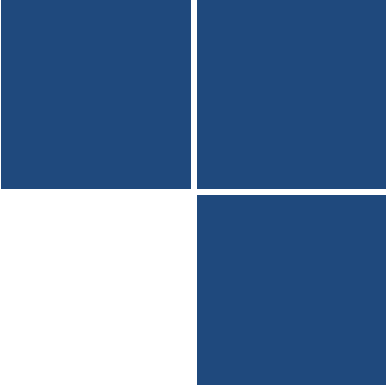
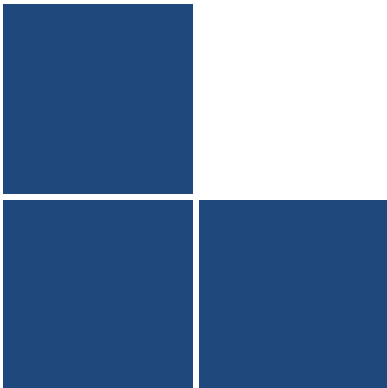




Plan 2024

Complejidad Ciclomática

Resoluciones



Ing. David CECCHI

UTN

Práctica - Complejidad Ciclomática

Para cada bloque de código fuente, se deberá representar su grafo correspondiente, calcular complejidad ciclomática asociada y plantear las rutas independientes necesarias para realizar las pruebas de caja blanca.

```
static void CondicionalV1(int a, int b)
{
    //condicional simple

    Console.WriteLine("Proceso Iniciado");

    if (a > b)
    {
        Console.WriteLine("Salida A");
    }
    else
    {
        Console.WriteLine("Salida B");
    }

    Console.WriteLine("Proceso Finalizado");
}
```

1

2

3

4

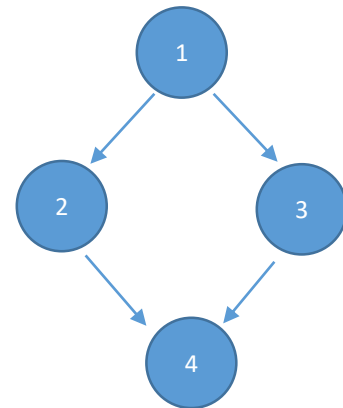
COMPLEJIDAD CICLOMATICA

 $V(G) = 2$ regiones $V(G) = 1$ nodo predicado + 1 = 2 $V(G) = 4$ aristas – 4 nodos + 2 = 2

RUTAS INDEPENDIENTES

Ruta 1: 1-2-4

Ruta 2: 1-3-4



```
static void CondicionalV2(int a, int b)
{
    //condicional anidado
    Console.WriteLine("Proceso Iniciado");
    if (a > b)
    {
        Console.WriteLine("Salida A");
    }
    else
    {
        if (b > a)
        {
            Console.WriteLine("Salida B");
        }
        else
        {
            Console.WriteLine("Salida Iguales");
        }
    }
    Console.WriteLine("Proceso Finalizado");
}
```

1

2

3

4

5

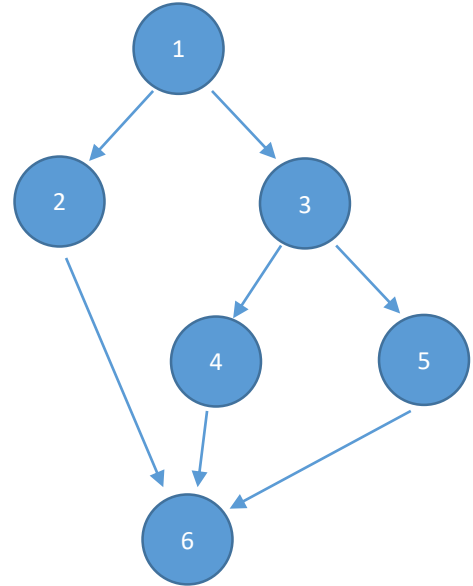
6

COMPLEJIDAD CICLOMATICA $V(G) = 3$ regiones $V(G) = 2$ nodo predicado + 1 = 3 $V(G) = 7$ aristas – 6 nodos + 2 = 3**RUTAS INDEPENDIENTES**

Ruta 1: 1-2-6

Ruta 2: 1-3-4-6

Ruta 3: 1-3-5-6



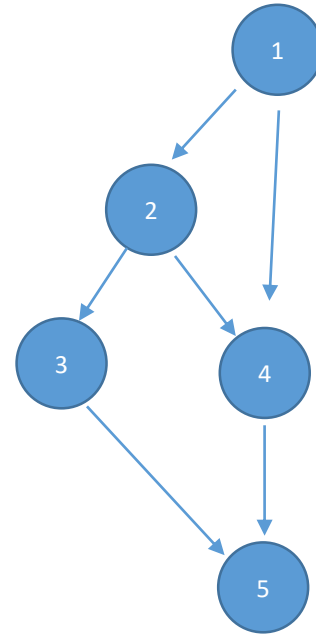
```
static void CondicionalV3(int a, int b)
{
    //condicional con condicion multiple
    1 Console.WriteLine("Proceso Iniciado");
    2 if (a % 2 == 0 && b % 2 == 0)
    {
        3 Console.WriteLine("Ambos valores Pares")
    }
    else
    {
        4 Console.WriteLine("Un valor Impar");
    }
    5 Console.WriteLine("Proceso Finalizado");
}
```

COMPLEJIDAD CICLOMATICA $V(G) = 3 \text{ regiones}$ $V(G) = 2 \text{ nodo predicado} + 1 = 3$ $V(G) = 6 \text{ aristas} - 5 \text{ nodos} + 2 = 3$ **RUTAS INDEPENDIENTES**

Ruta 1: 1-2-3-5

Ruta 2: 1-2-4-5

Ruta 3: 1-4-5



```
static void CondicionalV4(int a, int b)
{
    //condicional en serie
    Console.WriteLine("Proceso Iniciado");
    if (a > b)
    {
        Console.WriteLine("Salida A");
    }
    else
    {
        Console.WriteLine("Salida B");
    }
    if (a % 2 == 0)
    {
        Console.WriteLine("A es Par");
    }
    else
    {
        Console.WriteLine("A es Impar");
    }
    Console.WriteLine("Proceso Finalizado");
}
```

1

2

3

4

5

6

7

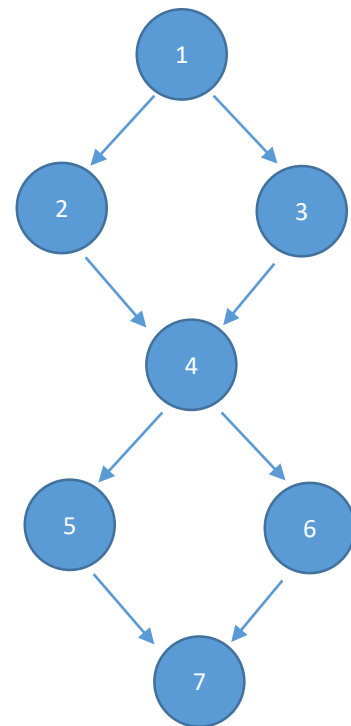
COMPLEJIDAD CICLOMATICA $V(G) = 3$ regiones $V(G) = 2$ nodo predicado + 1 = 3 $V(G) = 8$ aristas – 7 nodos + 2 = 3**RUTAS INDEPENDIENTES**

Ruta 1: 1-2-4-5-7

Ruta 2: 1-2-4-6-7

Ruta 3: 1-3-4-5-7

Ruta 4: 1-3-4-6-7 //no es linealmente independiente, no aporta nuevas aristas.
Podría optarse por descartar Ruta 3 e incluir la Ruta 4.



```
static void mientrasV1(int a, int b)
{
    //mientras simple

    int r = 0;
    while (r + a < b)
    {
        r += a;
    }

    Console.WriteLine($"Total: {r}");
}
```

1

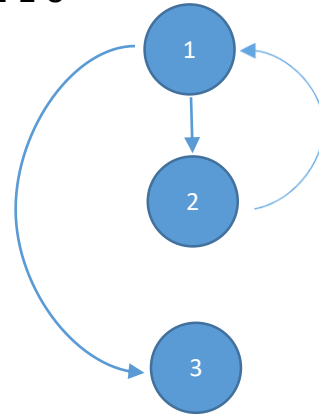
2

3

COMPLEJIDAD CICLOMATICA $V(G) = 2$ regiones $V(G) = 1$ nodo predicado + 1 = 2 $V(G) = 3$ aristas – 3 nodos + 2 = 2**RUTAS INDEPENDIENTES**

Ruta 1: 1-3

Ruta 2: 1-2-1-3



```
static void mientrasV2(int a, int b)
```

```
{
```

```
    //mientras simple y condicional
```

```
    int r = 0;
```

```
    int c = 0;
```

```
    while (r + a < b)
```

```
    {
```

```
        r += a;
```

```
        c++;
```

```
    }
```

```
    Console.WriteLine($"Total: {r}");
```

```
    if (c == 0)
```

```
    {
```

```
        Console.WriteLine("nunca suma");
```

```
    }
```

```
    else
```

```
    {
```

```
        Console.WriteLine($"ha sumado {c} veces");
```

```
    }
```

```
}
```

1

2

3

4

5

6

COMPLEJIDAD CICLOMATICA

$V(G) = 3$ regiones

$V(G) = 2$ nodo predicado + 1 = 3

$V(G) = 7$ aristas – 6 nodos + 2 = 3

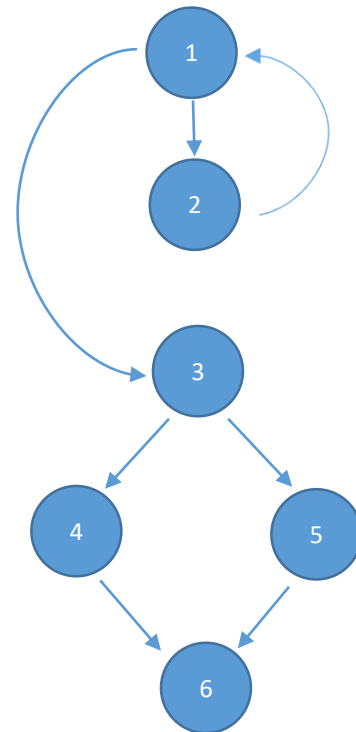
RUTAS INDEPENDIENTES

Ruta 1: 1-3-4-6

Ruta 2: 1-3-5-6

Ruta 3: 1-2-1-3-4-6

Ruta 4: 1-2-1-3-5-6 //no es linealmente independiente, no aporta nuevas aristas. Podría optarse por descartar Ruta 3 e incluir la Ruta 4.



```

static void mientrasV3(int a, int b)
{
    //mientras simple y condicionales en serie
    int r = 0;
    if (a > b)
    {
        Console.WriteLine("Salida A");
    }
    else
    {
        Console.WriteLine("Salida B");
    }
    if (a % 2 == 0)
    {
        Console.WriteLine("A es Par");
    }
    else
    {
        Console.WriteLine("A es Impar");
    }
    while (r + a < b)
    {
        r += a;
    }
    Console.WriteLine($"Total: {r}");
}

```

1

2

3

4

5

6

7

8

9

COMPLEJIDAD CICLOMATICA

V(G) = 4 regiones

V(G) = 3 nodo predicado + 1 = 4

V(G) = 11 aristas – 9 nodos + 2 = 4

ROUTAS INDEPENDIENTES

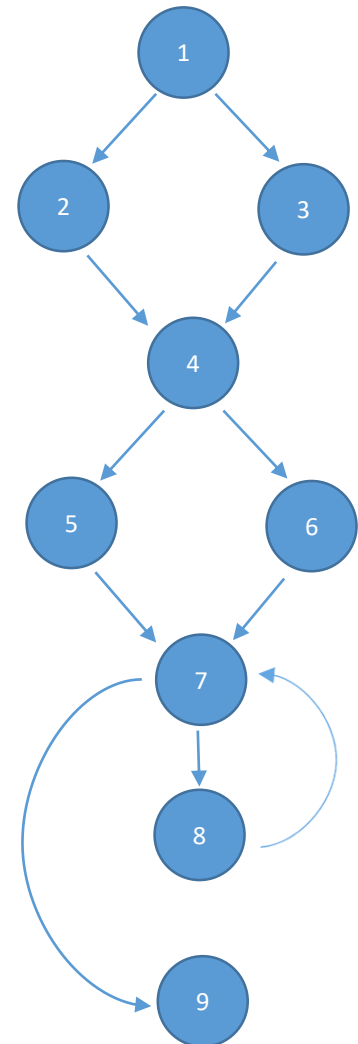
Ruta 1: 1-2-4-5-7-9

Ruta 2: 1-2-4-5-7-8-7-9

Ruta 3: 1-2-4-6-7-9

Ruta 4: 1-3-4-5-7-9

Ruta 5: 1-3-4-6-7-9 //no es linealmente independiente, no aporta nuevas aristas.
Podría optarse por descartar Ruta 4 e incluir la Ruta 5.




```
static void mientrasV4(int a, int b)
{
    //mientras en serie

    int ra = 0;
    int rb = 0;
    int ca = 0;
    int cb = 0;

    while (ra + a < b)
    {
        ra += a;
        ca++;
    }

    while (rb + b < a)
    {
        rb += b;
        cb++;
    }

    if (ca > cb)
    {
        Console.WriteLine("Salida A");
    }
    else
    {
        Console.WriteLine("Salida B");
    }
}
```

1

2

3

4

5

6

7

8

COMPLEJIDAD CICLOMATICA

$V(G) = 4$ regiones

$V(G) = 3$ nodo predicado + 1 = 4

$V(G) = 10$ aristas – 8 nodos + 2 = 4

RUTAS INDEPENDIENTES

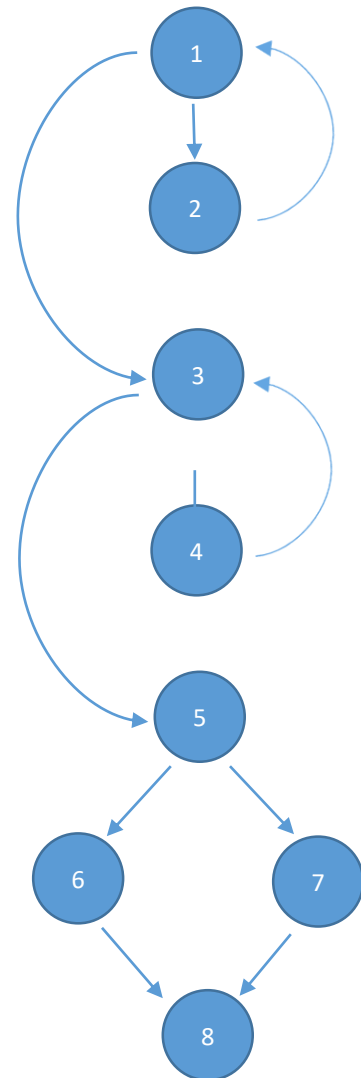
Ruta 1: 1-3-5-6-8

Ruta 2: 1-2-1-3-5-6-8

Ruta 3: 1-3-4-3-5-6-8

Ruta 4: 1-3-5-7-8

Ruta 5: 1-2-1-3-5-7-8 //no es linealmente independiente, no aporta nuevas aristas. Podría optarse por descartar Ruta 2 e incluir la Ruta 5.



```
static void mientrasV5(int a, int b)
{
    //mientras simple y condicional interno

    int r = 0, c = 0;
    while (r < b)
    {
        if (r+a < b)
        {
            r += a;
            c++;
        }
        else
        {
            r = b;
        }
    }
    Console.WriteLine($"Cantidad: {c}");
}
```

1

2

3

4

5

6

COMPLEJIDAD CICLOMATICA

$V(G) = 3$ regiones

$V(G) = 2$ nodo predicado + 1 = 3

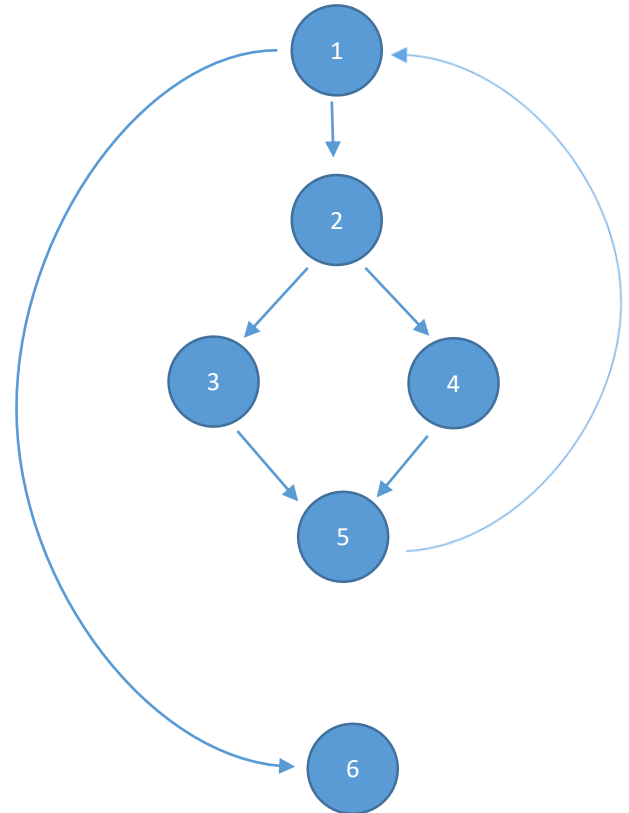
$V(G) = 7$ aristas – 6 nodos + 2 = 3

RUTAS INDEPENDIENTES

Ruta 1: 1-6

Ruta 2: 1-2-3-5-1-6

Ruta 3: 1-2-4-5-1-6



```
static float mientras(int[] valores, int minimo,
int maximo)
```

```
{
    int i, acumulado, validos;
    float promedio;

    i = 0;
    validos = 0;
    acumulado = 0;
    while (i < valores.Length && valores[i] != -99
    {
        if (valores[i] >= minimo && valores[i] <= maximo
        {
            validos++;
            acumulado += valores[i];

            i++;

            if (validos > 0)
            {
                promedio = acumulado / validos;
            }
            else
            {
                promedio = -999;
            }

            return promedio;
        }
    }
}
```

COMPLEJIDAD CICLOMATICA

$V(G) = 6$ regiones

$V(G) = 5$ nodo predicado + 1 = 6

$V(G) = 14$ aristas – 10 nodos + 2 = 6

RUTAS INDEPENDIENTES

Ruta 1: 1-7-9-10

Ruta 2: 1-7-8-10

Ruta 3: 1-2-7-8-10

Ruta 4: 1-2-3-6-1-7-8-10

Ruta 5: 1-2-3-4-6-1-7-8-10

Ruta 6: 1-2-3-4-5-6-1-7-8-10

Ruta 7: 1-2-3-4-5-6-1-7-9-10 //no es linealmente independiente, no aporta nuevas aristas. Podría optarse por descartar Ruta 6 e incluir la Ruta 7.

